Introduction to Blockchain Programming: Labs 01-03 report

Student: Muhammad Asfer (GitHub: MuhammadAsfer) Neptun: _____ Date: 25 Oct 2026
Repository: github.com/MuhammadAsfer/OE_BLOCKCHAIN_Labs

Environment: Windows, Docker Desktop, Bitcoin Core (regtest), Python 3.14 via uv. All code is in my own repository; reference tests: txparser 19, Lab 02 27, Lab 03 17 passed (my tests, built from the published vectors).

Lab 01 - Bitcoin regtest and raw transactions

Setup

The node started on regtest with 0 blocks. The setup script failed on this machine (it tried to load the wallet before it existed), so I did its steps by hand: `createwallet lab01`, a mining address, and `generatetoaddress 101`. Result: `getbalance = 50.00000000`.

Part A - UTXO state

Command	Observed
listunspent	1 output, 50 BTC, txid acfbc3ed1fa82ccc...742d1c7a, vout 0, 101 confirmations, spendable
getbalances	trusted 50, immature 5000
gettxoutsetinfo	height 101, txouts 101, transactions 101, total_amount 5050

Why the numbers differ. The node counts every unspent output regardless of owner or maturity; the wallet only shows what it can spend now. Both agree underneath: 101 coinbase outputs of 50 BTC exist and exactly one is mature (101 - 100 blocks of maturity). `txouts` is 101, not 202, because each coinbase also carries a zero-value OP_RETURN witness commitment, which is provably unspendable and never enters the UTXO set.

Part B - the transaction

Input: 50 BTC. Payment: 10 BTC to `bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3`. Fee chosen: 0.0001 BTC. Change = 50 - 10 - 0.0001 = **39.9999 BTC** to `bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl`. txid: `b95d32f3...e3e8d2b5`, mined in block 102.

decoderawtransaction:


```
{
  "txid": "b95d32f37c0aee53c76e9abb90546663f9c7e228d3fc67badb3f2cd0c3e8d2b5",
  "hash": "dd709d1d3d4d82ca5c1d6c77d9fd3e780f291a172cd9f771063d54e96876a76d",
  "version": 2,
  "size": 222,
  "vsize": 141,
  "weight": 561,
  "locktime": 0,
  "vin": [
    {
      "txid": "acfbc3ed1fa82cccddc03c26f4b62f338d1e9790ced1b0eb5d9ffeca742d1c7a",
      "vout": 0,
      "scriptSig": {
        "asm": "",
        "hex": ""
      },
      "txinwitness": [
        "304402200b5b45611155fef8016925a55235e341be5a8bd58c1c0110ad50a3c997fa22b102203a177e880bc5761f14de299d1aab7d49a44f52439e3
    eb40e9edca0c54a0e247801",
        "029df6882963c5dbdc3097ba155841deb451a85c0e65b0280ef1d2739fa8a55334"
      ],
      "sequence": 4294967293
    }
  ],
  "vout": [
    {
      "value": 10.00000000,
      "n": 0,
      "scriptPubKey": {
        "asm": "0 25b136b8cc9fd803cc11bd4115942e66dc7fedec",
        "desc": "addr(bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3)#z86v0dxc",
        "hex": "001425b136b8cc9fd803cc11bd4115942e66dc7fedec",
        "address": "bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3",
        "type": "witness_v0_keyhash"
      }
    },
```

```
    {
      "value": 39.99990000,
      "n": 1,
      "scriptPubKey": {
        "asm": "0 bd261d35ff7baf5ce3f22b3592dbfdc8fb9f143c",
        "desc": "addr(bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl)#0jehjwf3",
        "hex": "0014bd261d35ff7baf5ce3f22b3592dbfdc8fb9f143c",
        "address": "bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl",
        "type": "witness_v0_keyhash"
      }
    }
  ]
}
```


gettransaction (fee -0.0001, 2 confirmations at the time, bip125-replaceable: no because the transaction was already mined):


```
{
  "amount": 0.00000000,
  "fee": -0.00010000,
  "confirmations": 2,
  "blockhash": "75fbfceb53b068f4ea64ea9243df23af236d520be6fc00465616c855075577ac",
  "blockheight": 102,
  "blockindex": 1,
  "blocktime": 1792928920,
  "txid": "b95d32f37c0aee53c76e9abb90546663f9c7e228d3fc67badb3f2cd0c3e8d2b5",
  "wtxid": "dd709d1d3d4d82ca5c1d6c77d9fd3e780f291a172cd9f771063d54e96876a76d",
  "walletconflicts": [
  ],
  "time": 1792928899,
  "timereceived": 1792928899,
  "bip125-replaceable": "no",
  "details": [
    {
      "address": "bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3",
      "category": "send",
      "amount": -10.00000000,
      "label": "destination",
      "vout": 0,
      "fee": -0.00010000,
      "abandoned": false
    },
    {
      "address": "bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl",
      "category": "send",
      "amount": -39.99990000,
      "label": "change",
      "vout": 1,
      "fee": -0.00010000,
      "abandoned": false
    },
    {
      "address": "bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3",
      "parent_descs": [
        "wpkh(tpubD6NzVbkrYhZ4XVWwRqhbZzvfsEJp3HNYqp94Hvf2twLtTq7u1wxeho7g6RFQy9Y6sj1wkpjPDxsWYPKxMjuhWhD2eEkZ555jxMDoaExrZXm/84
    h/1h/0h/0/*)#l8s5srn5"
      ],
      "category": "receive",
      "amount": 10.00000000,
      "label": "destination",
      "vout": 0,
      "abandoned": false
    },
    {
      "address": "bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl",
      "parent_descs": [
        "wpkh(tpubD6NzVbkrYhZ4XVWwRqhbZzvfsEJp3HNYqp94Hvf2twLtTq7u1wxeho7g6RFQy9Y6sj1wkpjPDxsWYPKxMjuhWhD2eEkZ555jxMDoaExrZXm/84
    h/1h/0h/0/*)#l8s5srn5"
      ],
      "category": "receive",
      "amount": 39.99990000,
      "label": "change",
      "vout": 1,
      "abandoned": false
    }
  ],
  "hex": "020000000001017a1c2d74cafe9f5debb0d1ce90971e8d332fb6f4263cc0ddcc2ca81fedc3fbac0000000000fdffffff0200ca9a3b000000001600
    1425b136b8cc9fd803cc11bd4115942e66dc7fedecf0006bee00000000160014bd261d35ff7baf5ce3f22b3592dbfdc8fb9f143c0247304402200b5b4561
    1155fef8016925a55235e341be5a8bd58c1c0110ad50a3c997fa22b102203a177e880bc5761f14de299d1aab7d49a44f52439e3eb40e9edca0c54a0e2478
    0121029df6882963c5dbdc3097ba155841deb451a85c0e65b0280ef1d2739fa8a5533400000000",
  "lastprocessedblock": {
    "hash": "7c42102616edea8e9f470bc88fd35fbd162456a8db400c66b2aa2f7d950cb15d",
    "height": 103
  }
}
```


... (details and hex trimmed)

Checks. size 222, vsize 141, weight 561 = 3 x 113 + 222; fee rate = 10 000 sat / 141 vB = 70.9 sat/vB. Balance after mining: 99.9999 BTC (block 102 matured the second coinbase). txid differs from hash (wtxid) because the transaction is SegWit: the txid hashes the legacy serialisation without witness data, which is the malleability fix.

Why did createrawtransaction not complain? It is only a serialiser; validation happens at broadcast, where a transaction spending more than it has would be rejected with bad-txns-in-belowout.

Part C - my own parser

```
$ uv run python -m txparser <signed hex>
txid      : b95d32f37c0aee53c76e9abb90546663f9c7e228d3fc67badb3f2cd0c3e8d2b5
wtxid     : dd709d1d3d4d82ca5c1d6c77d9fd3e780f291a172cd9f771063d54e96876a76d
version   : 2   locktime: 0   segwit: True
size      : 222   stripped: 113   weight: 561   vsize: 141
input  0  : acfbc3ed1fa82cccddc03c26f4b62f338d1e9790ced1b0eb5d9ffeca742d1c7a:0  sequence=0xfffffffd
  witness : 2 items, sizes [71, 33]
output 0  : 1000000000 sat = 10.00000000 BTC  p2wpkh  OP_0 25b136b8cc9fd803cc11bd4115942e66dc7fedec
  address : bcrt1qykcndwxvnlvq8nq3h4q3t9pwvmw8lm0vnylts3
output 1  : 3999990000 sat = 39.99990000 BTC  p2wpkh  OP_0 bd261d35ff7baf5ce3f22b3592dbfdc8fb9f143c
  address : bcrt1qh5np6d0l0wh4eclj9v6e9klaerae79pu499enl
fee       : unknown (needs the values of the spent outputs)
```

Every field equals the node's decode: txid, wtxid (= *hash*), sizes, sequence 0xfffffffd (RBF signalled), a 71-byte DER signature plus 33-byte compressed public key in the witness, and two p2wpkh outputs. `uv run pytest -v`: 19 passed (CompactSize edge cases, genesis and SegWit vectors, script classifier, address vectors, rejection of truncated/trailing/non-hex input).

Why input 0 starts at byte 7: 4 (version) + 2 (marker 00, flag 01) + 1 (input count) = 7; legacy has no marker/flag so it starts at 5. **Why the parser cannot print the fee:** an input only names txid:vout; its value lives in the earlier transaction, so the sum of inputs is unknown without a lookup.

Part D - the deliberate mistake

I spent the block-2 coinbase (txid f6a4b197...6557b, 50 BTC) with a single 10 BTC output and no change. Result:

The first `sendrawtransaction` was rejected by the node with **error code -25**: the transaction pays a fee far above the RPC `maxfeerate` limit (default 0.10 BTC/kvB). The same transaction was accepted only after I passed `maxfeerate = 0`, which shows the rejection came from the RPC safety limit and not from consensus.

Retrying with `maxfeerate = 0` returned txid d93d571a...5183ce6. After mining, block 103 contained: **tx[0] (coinbase) vout[0].value = 90.00000000** (50 subsidy + 40 fee), vout[1] = 0 (witness commitment), and tx[1] = my transaction with one output of 10 BTC and `"fee": 40.00000000`. The 40 BTC went to the miner of the block, here my own wallet as an immature coinbase; on mainnet it would be a stranger.

Is -25 a consensus rule? No. It is a safety limit in the node's RPC; the network accepts the transaction. Consensus only requires sum(inputs) >= sum(outputs) and a coinbase claiming at most subsidy + fees. **Why this cannot happen on Ethereum:** accounts, not outputs. The state transition subtracts value + gasUsed x price from the sender and anything unspent stays in the account; the maximum fee is an explicit signed bound (gasLimit x maxFeePerGas).

Lab 02 - Keys and cryptography

`uv run pytest`: 27 passed (3.8 s). Tests use published vectors: 2G, the BIP-39 zero-entropy seed, three Ethereum addresses of the abandon...about mnemonic, the four EIP-55 examples, Keccak-256 of the empty string, the RFC 6979 vector (key 1, "Satoshi Nakamoto"), and the BIP-84 address bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu.

Output of `uv run python -m crypto_lab`:

```
== Part A: curve ==
G on curve : True
2G x       : c6047f9441ed7d6d3045406e95c07cd85c778e4b8cef3ca7abac09b95c709ee5
n*G = O    : True
P+(-P) = O : True
ladder == double-and-add: True
SEC1 round trip: True

== Part B: ECDSA ==
address    : 0x97a10005a72997fa009eA4366B38702A3A2f500F
verify     : True
(r, n-s) also verifies: True
deterministic: True
recover pubkey: True
identical r: True
original d : 0xc0ffee00112233445566778899aabbccddeeff00112233445566778899aabbcc
recovered d: 0xc0ffee00112233445566778899aabbccddeeff00112233445566778899aabbcc -> match: True

== Part C: HD wallet ==
seed       : 5eb00bbddcf06908...
m/44'/60'/0'/0/0: 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
m/44'/60'/0'/0/1: 0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0
m/44'/60'/0'/0/2: 0xb6716976A3ebe8D39aCEB04372f22Ff8e6802D7A
passphrase 'correct horse'  : 0xFf0Bdf99513306994d4aF9949BA7cfA60204D285
passphrase 'correct horse ' : 0x67dA015b6A80F19822e5d270A7788F72f018a724
parent key recovered from child + xpub: True
watch-only derivation matches: True
hardened refused: hardened child: I_L cannot be computed without the parent private key

== Part D: post-quantum cost ==
scheme                  pk   sig   tx B  x ECDSA    tps
ECDSA (secp256k1)       33    72    215     1.0x   7.75
Schnorr (BIP-340)       32    64    206     1.0x   8.09
FN-DSA-512             897   666   1673     7.8x   1.00
ML-DSA-44             1312  2420   3842    17.9x   0.43
ML-DSA-65             1952  3309   5371    25.0x   0.31
SLH-DSA-128s            32  7856   7998    37.2x   0.21

Exposure report:
  p2pkh_unspent         PROTECTED  only HASH160(pubkey) is public until the coin is spent
  p2pk                  EXPOSED    the full public key sits in the output
  reused_address        EXPOSED    an earlier spend already revealed the public key
  mempool_transaction   EXPOSED    the spend reveals the key while it waits for confirmation
  ethereum_any_active   EXPOSED    ecrecover derives the public key from any past signature
```

Explanations

Division = modular inverse. A finite field has no fractions; a/b means a x b^{-1} mod p, computed with Fermat as b^{p-2} mod p. Python's / returns a float, so the result silently falls off the curve.

Double-and-Add vs ladder. Double-and-Add adds only when the current key bit is 1, so timing/power reveals the bits (extra additions = Hamming weight - 1). The Montgomery ladder does one add and one double per bit regardless of value; the test that both give identical results on random scalars shows the hardened version is functionally the same. (Python big-integer arithmetic is itself not constant-time, so this only demonstrates the idea.)

Nonce reuse. Same k gives the same r and two equations s1k = z1 + rd, s2k = z2 + rd. Subtracting: k = (z1 - z2)/(s1 - s2), then d = (s1k - z1)/r. The demo recovered the exact private key. Because low-s may have negated either s (flipping s equals negating k), the code tries all four sign combinations and keeps the d whose d*G equals the known public key.

RFC 6979 derives k = HMAC-DRBG(d, z): unique per message so r never repeats under one key, and not computable without d, so it fixes reuse without making k public. **Low-s:** (r, s) and (r, n-s) both verify (shown in the output), which is signature malleability; Bitcoin policy and Ethereum EIP-2 accept only s <= n/2.

HD wallets. The passphrase is part of the PBKDF2 salt, so every passphrase yields a valid wallet: 'correct horse' and 'correct horse ' (one space) give unrelated addresses, so a typo is indistinguishable from theft. For a normal child I_L depends only on public data (parent public key + chain code, both in an xpub), so parent = child - I_L: the output shows the parent key recovered from one child plus the xpub, and refused for a hardened child. BIP-44 hardens purpose, coin type and account to stop the attack climbing above the account. Ethereum uses

Keccak-256, not NIST SHA3-256 (different padding), and EIP-55 capitalises letters by the Keccak hash of the lowercase hex string.

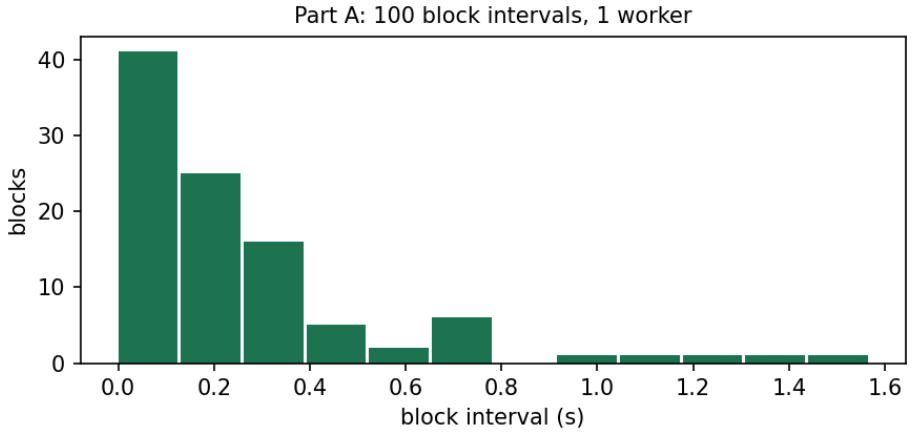
Part D question - migrate now or later? Now. The ledger is permanent, so public keys published today can be collected and broken whenever quantum hardware arrives (harvest now, break later). Unspent P2PKH is protected (only HASH160 is public) but P2PK, reused addresses, mempool transactions and every Ethereum account that has sent a transaction are exposed. If assets must stay safe for 15 years and migration takes years, the decision is already late. Cost: ML-DSA-44 makes a one-input transaction 3 842 B (17.9x ECDSA) and cuts throughput from 7.75 to 0.43 tx/s at 1 MB / 10 min.

Lab 03 - Consensus simulation

`uv run pytest`: 17 passed.

Part A - Proof of Work (single worker)

```
== Part A: mining 100 blocks, workers=1 ==
mean 0.263s  median 0.170s  stdev 0.300s
median/mean 0.647 (theory 0.693)   stdev/mean 1.140 (theory 1.0)
min 0.0001s  max 1.571s  span 11078x
retarget(1000,300,600) = 2000  (1000,1200,600) = 500  (1000,6,600) = 4000  (1000,60000,600) = 250
histogram (12 bins from 0 to max):
    0.00s | ########################################
    0.13s | ########################
    0.26s | ################
    0.39s | #####
    0.52s | ##
    0.65s | ######
    0.79s |
    0.92s | #
    1.05s | #
    1.18s | #
    1.31s | #
    1.44s | #
```



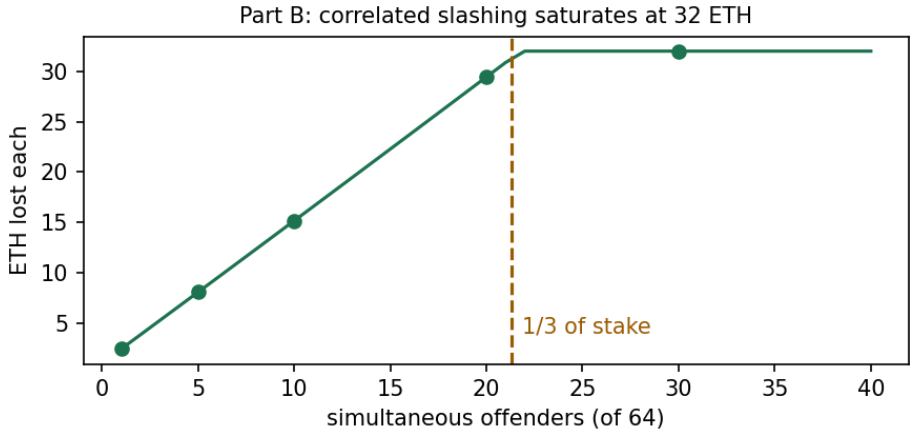
The histogram's tallest bar is the leftmost and it decays roughly geometrically (not a bell curve). median/mean = 0.647 (theory ln 2 = 0.693) and stdev/mean = 1.14 (theory 1.0); the largest interval is thousands of times the smallest. With 100 blocks the ratios are noisy, which matches 1/sqrt(N). **Why --workers 1:** mine_parallel spawns new processes per block; start-up (~1 s with spawn) exceeds the ~0.3 s mining time, so several workers measure process start-up, giving nearly constant intervals. **Why median < mean:** every hash is an independent trial, so waiting time is geometric/exponential, a right-skewed distribution where rare long gaps pull the mean up; half the intervals are below 0.693 x mean. This memorylessness also makes mining fair: a miner with 10% of the hash rate wins 10% of blocks.

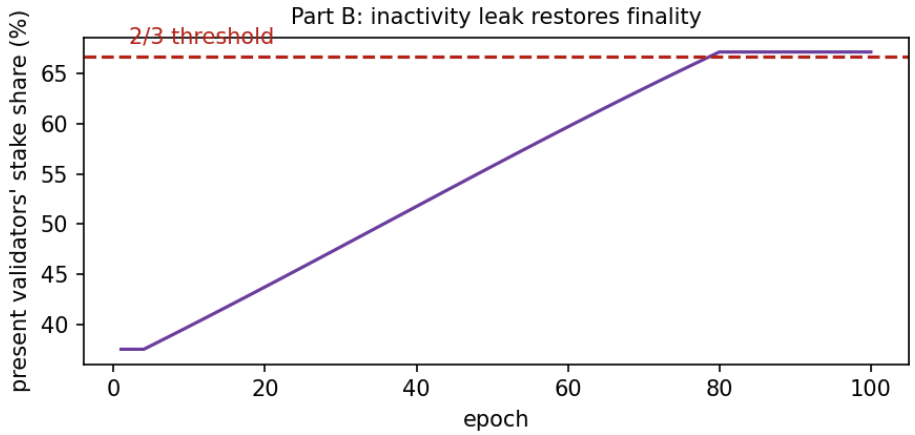
Retargeting. retarget(1000,300,600) = 2000, (1000,1200,600) = 500, (1000,6,600) = 4000, (1000,60000,600) = 250: the ratio is clamped to [1/4, 4] so one manipulated or catastrophic window cannot push difficulty somewhere the chain cannot recover from. A long window (Bitcoin: 2016 blocks) is needed because the hash-rate estimate from N exponential intervals has relative error about 1/sqrt(N): ~100% for one block, ~2% for 2016.

Part B - Casper FFG

```
== Part B: Casper FFG ==
-- Scenario 1: finality
  epoch 1: justified Checkpoint(epoch=1, root='root-1')  finalized Checkpoint(epoch=0, root='genesis')
  epoch 2: justified Checkpoint(epoch=2, root='root-2')  finalized Checkpoint(epoch=1, root='root-1')
  epoch 3: justified Checkpoint(epoch=3, root='root-3')  finalized Checkpoint(epoch=2, root='root-2')
-- Scenario 2: one equivocator: loss each 2.4084 ETH
-- Scenario 3: twenty at once: loss each 29.4320 ETH
-- Scenario 4: surround vote -> surround_vote
-- Slashing table (64 validators):
    1 offenders ->   2.41 ETH each
    5 offenders ->   8.06 ETH each
   10 offenders ->  15.15 ETH each
   20 offenders ->  29.43 ETH each
   30 offenders ->  32.00 ETH each
-- Scenario 5: inactivity leak (24 of 64 present, 100 epochs)
   epoch   1: voting share  37.5%  leaking=False  justified=False  finalized epoch=0
   epoch   5: voting share  37.9%  leaking=True  justified=False  finalized epoch=0
   epoch  11: voting share  40.2%  leaking=True  justified=False  finalized epoch=0
   epoch  20: voting share  43.7%  leaking=True  justified=False  finalized epoch=0
```

```
epoch  40: voting share  51.7%  leaking=True  justified=False  finalized epoch=0
epoch  60: voting share  59.7%  leaking=True  justified=False  finalized epoch=0
epoch  80: voting share  67.2%  leaking=True  justified=True  finalized epoch=0
epoch  82: voting share  67.2%  leaking=False  justified=True  finalized epoch=81
epoch  90: voting share  67.2%  leaking=False  justified=True  finalized epoch=89
epoch 100: voting share  67.2%  leaking=False  justified=True  finalized epoch=99
```





Justify/finalise. A checkpoint is justified by a link from an already justified source with strictly more than 2/3 of active stake (with 9 equal validators 6 do not justify, 7 do; with 64, 43 are needed) and finalised when its immediate successor is justified too. Strictly more than 2/3 because two conflicting supermajorities then overlap in more than 1/3 of the stake, who must have voted twice and are slashable. **Slashing:** initial penalty = balance/32, then a correlated penalty balance x min(1, 3 x slashed/total). The same offence costs 2.41 ETH alone but 29.43 ETH when 20 validators do it together, because simultaneous equivocation of a third of the stake is what could actually finalise two histories. The curve saturates at 32 ETH near 1/3 of the stake. **Inactivity leak:** with only 24 of 64 validators attesting nothing finalises (CAP: safety over liveness; the absent third might be finalising something else). After a 4-epoch grace period absent validators lose 1.6% of balance per epoch until present validators exceed 2/3 (about epoch 80), then justification resumes and finalisation follows.

Part C - gossip, partition, heal (12 nodes, three seeds)

```
== Part C: gossip, partition, heal ==
connected phase : tips=1
partitioned     : distinct tips=2  heights=[7, 11]
after heal      : distinct tips=1  heights=[16]  converged=True
reorg depths per node: {'n0': [1], 'n1': [1, 1], 'n2': [], 'n3': [1], 'n4': [1], 'n5': [1], 'n6': [2], 'n7': [2], 'n8': [2], 'n9
    ': [2, 1], 'n10': [2, 1], 'n11': [2, 1]}
== Part C: gossip, partition, heal ==
connected phase : tips=1
partitioned     : distinct tips=2  heights=[8, 13]
after heal      : distinct tips=1  heights=[19]  converged=True
reorg depths per node: {'n0': [], 'n1': [], 'n2': [], 'n3': [1], 'n4': [1], 'n5': [], 'n6': [4], 'n7': [4], 'n8': [4], 'n9': [4]
    , 'n10': [4, 1], 'n11': [4]}
== Part C: gossip, partition, heal ==
connected phase : tips=1
partitioned     : distinct tips=2  heights=[8, 9]
after heal      : distinct tips=1  heights=[15]  converged=True
reorg depths per node: {'n0': [1, 3], 'n1': [1, 3], 'n2': [3], 'n3': [3], 'n4': [3], 'n5': [2, 3], 'n6': [1, 2, 3], 'n7': [2, 3]
    , 'n8': [2, 3], 'n9': [], 'n10': [], 'n11': [1, 2, 3]}
```

During each split there were two distinct tips with different heights; after healing every run converged to a single tip. The lighter side reorganised with depth 2-4 (and a few depth-1 reorgs elsewhere), and transactions confirmed only in orphaned blocks return to the mempool (_prune_mempool keeps only transactions in the canonical chain): probabilistic finality. Both halves were 'right' during the split because each built a valid chain on a valid parent and could not see the other; a node cannot tell a partitioned peer from a slow one. The tip moves only on strictly greater work, so two tips at the *same* height after a heal would be a legitimate tie (one earlier run showed this), while different heights would be a bug. Results differ between runs because asyncio scheduling depends on real time.

Bug found while building: my first block header did not commit to the miner, so two different blocks hashed identically; the equal-work reorg test caught it. A header must commit to its coinbase/contents, as Bitcoin's does through the Merkle root.

Honest scope note: the 'Hard' practice items, the 'Beyond the labs' challenges and the bonuses are not part of this submission.